

Apply Filters to SQL Queries

Portfolio Activity Document | SQL Filtering for Security Investigation

Project Description

This portfolio document demonstrates how SQL filters can be used to investigate potential security issues involving login activity and employee devices. In this scenario, I used queries on the **log_in_attempts** and **employees** tables to find failed logins, review suspicious dates, identify login attempts outside of Mexico, and locate employee machines that require security updates. The work shows how SQL operators such as **AND**, **OR**, **NOT**, and **LIKE** can help narrow large datasets into focused results for security analysis.

Database Tables Used

The activity uses two database tables. The **log_in_attempts** table stores login event information, including event ID, username, login date, login time, country, IP address, and whether the login attempt was successful. The **employees** table stores employee and device information, including employee ID, device ID, username, department, and office location.

Table	Relevant Columns	Security Purpose
log_in_attempts	login_date, login_time, country, success, ip_address, username	Investigate suspicious login activity and failed access attempts.
employees	employee_id, device_id, username, department, office	Identify employee machines for security updates by department and location.

1. Retrieve After-Hours Failed Login Attempts

The first task was to identify failed login attempts that occurred after business hours. Since the activity defines after hours as after 18:00 and the **success** column stores failed attempts as **FALSE**, the query filters for both conditions using **AND**.

```
SELECT *
FROM log_in_attempts
WHERE login_time > '18:00'
AND success = FALSE;
```

This query returns only records where the login attempt happened after 6:00 p.m. and the login attempt failed. Using **AND** is important because both conditions must be true for the record to be relevant to the investigation.

2. Retrieve Login Attempts on Specific Dates

The next task was to review login attempts that occurred on the suspicious date, 2022-05-09, and on the day before, 2022-05-08. The query uses **OR** because records from either date should be included.

```
SELECT *
FROM log_in_attempts
WHERE login_date = '2022-05-09'
OR login_date = '2022-05-08';
```

This query returns all login attempts from both dates. The **OR** operator expands the results so the security team can compare activity from the suspicious date with activity from the previous day.

3. Retrieve Login Attempts Outside of Mexico

The security team determined that the suspicious activity did not originate in Mexico. Because the country field may contain values such as **MEX** or **MEXICO**, the query uses **NOT** with **LIKE** and the wildcard % to exclude any country value that begins with MEX.

```
SELECT *
FROM log_in_attempts
WHERE NOT country LIKE 'MEX%';
```

This query returns login attempts from countries outside of Mexico. The **LIKE** pattern **MEX%** matches both MEX and MEXICO, while **NOT** removes those records from the result set.

4. Retrieve Employees in Marketing

The team needed to identify employee machines in the Marketing department located in the East building. The query uses **AND** because the employee must be in Marketing and their office must begin with East.

```
SELECT *
FROM employees
WHERE department = 'Marketing'
AND office LIKE 'East%';
```

This query returns employees who match both criteria. The **LIKE** operator with **East%** includes offices such as East-170 and East-320, while excluding offices in other buildings.

5. Retrieve Employees in Finance or Sales

The next security update applied to employee machines in either the Finance department or the Sales department. The query uses **OR** so that employees from either department are included.

```
SELECT *  
FROM employees  
WHERE department = 'Finance'  
OR department = 'Sales';
```

This query returns all employees who work in Finance or Sales. The **OR** operator is appropriate because the employee only needs to belong to one of the listed departments to be included in the results.

6. Retrieve All Employees Not in IT

The final task was to identify employees who still needed a security update. Since employees in the Information Technology department already received the update, the query excludes that department using **NOT**.

```
SELECT *  
FROM employees  
WHERE NOT department = 'Information Technology';
```

This query returns employees from every department except Information Technology. The result helps the security team focus the update process on employee machines that still require attention.

Summary

This activity demonstrated how SQL can support security investigations by filtering login and employee data into useful results. I used **AND** to require multiple conditions, **OR** to include records that match either condition, **NOT** to exclude unwanted results, and **LIKE** with wildcards to search for patterns. These queries helped identify after-hours failed login attempts, suspicious login dates, activity outside of Mexico, and employee devices that required security updates. This type of filtering is useful for narrowing security data, supporting investigations, and improving response efficiency.

Skills Demonstrated

- Used SQL filters to investigate security-related records.
- Applied AND and OR operators to combine multiple conditions.
- Used NOT to exclude records from query results.
- Used LIKE and % wildcard matching to search for patterns in text fields.
- Filtered date, time, department, country, and office-location data.
- Connected SQL query results to practical security investigation tasks.

Reference Notes

This document was completed using the provided SQL query instruction document, the portfolio activity template, and the table format reference for the organization database. Typed SQL queries are shown in monospaced gray code blocks to follow the formatting guidance for portfolio-ready command and query examples.